arXiv:2509.08753v2 [cs.CL] 29 Sep 2025

Streaming Sequence-to-Sequence Learning with Delayed Streams Modeling

Neil Zeghidour* **Eugene Kharitonov** **Manu Orsini**
Václav Volhejn **Gabriel de Marmiesse** **Edouard Grave**
Patrick Pérez **Laurent Mazaré** **Alexandre Défossez***
*Equal contribution
Kyutai
{neil, eugene, alex}@kyutai.org

Abstract

We introduce Delayed Streams Modeling (DSM), a flexible formulation for streaming, multimodal sequence-to-sequence learning. Sequence-to-sequence generation is often cast in an offline manner, where the model consumes the complete input sequence before generating the first output timestep. Alternatively, streaming sequence-to-sequence rely on learning a policy for choosing when to advance on the input stream, or write to the output stream. DSM instead models already time-aligned streams with a decoder-only language model. By moving the alignment to a pre-processing step, and introducing appropriate delays betweem streams, DSM provides streaming inference of arbitrary output sequences, from any input combination, making it applicable to many sequence-to-sequence problems. In particular, given a text and audio stream, automatic speech recognition (ASR) corresponds to the text stream being delayed, while the opposite gives a text-to-speech (TTS) model. We perform extensive experiments for these two major sequence-to-sequence tasks, showing that DSM provides state-of-the-art performance and latency while supporting arbitrary long sequences, being even competitive with offline baselines. Code, samples and demos are available at github.com/kyutai-labs/delayed-streams-modeling.

1 Introduction

We are interested in streaming sequence-to-sequence (seq2seq) learning, i.e. predicting an output sequence as we process an input sequence synchronously, as opposed to offline seq2seq where inputs are recorded entirely before producing the output sequence. The latter class of offline models was introduced for a diverse set of tasks such as handwriting recognition (Graves et al., 2013), automatic speech recognition (ASR) (Graves et al., 2013) or machine translation (Bahdanau et al., 2015; Sutskever et al., 2014), by designing modality-dependent input encoders, typically coupled with a text decoder (Hochreiter & Schmidhuber, 1997). Although this asymmetry between input processing and output generation facilitated the adoption of this framework in many tasks, it also led to a divergence of model architectures across modalities. As an example, a Tacotron text-to-speech (TTS) model (Wang et al., 2017) would differ from an ASR model such as LAS (Chan et al., 2016). The advent of decoder-only Transformers (Vaswani et al., 2017; Radford et al., 2018) for text language modeling reduced the gap between input and output processing by allowing a single model to process a simple concatenation of tokens. In parallel, neural compression algorithms that can transform images (Razavi et al., 2019; Esser et al., 2020) and audio (Zeghidour et al., 2022; Défossez et al., 2023) into discrete tokens analogous to text allowed integrating these modalities along text sequences. Thus, a decoder-only model can be used for seq2seq tasks such as ASR (Rubenstein et al., 2023), TTS (Wang et al., 2023; Kharitonov et al., 2023), spoken dialogue (Défossez et al., 2024), visual

Preprint. Under review.

understanding (Beyer et al., 2024) or image generation (Ramesh et al., 2021). Furthermore, inputs and outputs are interchangeable in this framework, meaning a single model can be trained for generation in both directions: AudioPALM (Rubenstein et al., 2023) performs TTS and ASR, while CM3Leon (Yu et al., 2023) provides both image captioning and generation. Yet, a major limitation of these decoder-only approaches is their incompatibility with streaming. First, their prefix-based formulation requires access to the full input sequence before generation, which prevents real-time inference and inherently limits the maximum input length. Second, modalities operate at differing framerates: audio or video tokens are typically sampled regularly, while text tokens represent linguistic units pronounced with varying durations. This prevents applications such as meeting transcription or continuous translation.

Another popular approach is to learn an alignment policy between modalities, using architectures such as Transducers (Graves, 2012; Zhang et al., 2020), or specific attention formulations (Raffel et al., 2017; Guo et al., 2024). At inference, the policy decision will change which modules to execute at each step, which is detrimental to batching. Besides, learning the policy requires train-time exploration, which can be costly. As noted by Ma et al. (2019), a simple *wait-k* policy can be used, especially for same modality sequence-to-sequence modeling.

In this work, we present Delayed Streams Modeling (DSM), a framework for streaming sequence-to-sequence learning across modalities. We make a simplifying assumption compared with previous *wait-k* based methods (Ma et al., 2021; Chen et al., 2021b), namely that both modalities are aligned to a shared framerate as a pre-processing step. DSM uses a decoder-only model to process as many parallel token streams as there are I/O sequences. This multistream architecture, introduced by Défossez et al. (2024), allows for a synchronous autoregressive modeling of aligned sequences which—when coupled with a finite context—provides real-time, streaming generation over infinite input sequences. Moreover, by operating at a constant framerate, DSM allows for batching, a feature rarely provided by streaming models. The second key component of DSM, inspired by the *wait-k* policy (Ma et al., 2019), is the introduction of a delay between streams to control the quality/latency trade-off: shifting a sequence B such that it is delayed w.r.t. sequence A allows for a better prediction of the former based on the latter. With appropriate delays, a DSM model can be trained to continuously predict any combination of output sequences from any combination of input sequences. To illustrate the abilities of the DSM framework, we train speech-text models for ASR and TTS. We show how DSM provides a state-of-the-art tradeoff between latency—as low as a few hundred milliseconds—and quality, while providing long-form synthesis and transcription, along with precise word timestamps that locate where they are pronounced.

2 Related Work

Streaming Sequence-to-Sequence Learning. Most streaming seq2seq literature has focused on speech-to-text tasks, in particular ASR (Li et al., 2021) and translation (Xue et al., 2023; Barrault et al., 2023). Monotonic (Raffel et al., 2017; Chiu & Raffel, 2018) and local (Chiu et al., 2019) attention respectively allow for causal attention of outputs with respect to inputs along with handling arbitrarily long sequences. A common limitation of streaming models is their incompatibility with batching when using an inference policy (Barrault et al., 2023), or the lack of symmetry meaning that specific models must be used for speech-to-text (Li et al., 2021) and text-to-speech (Wang et al., 2017). Previous approaches using Transformer decoder-only models (Guo et al., 2024; Chen et al., 2024a) typically require non-standard attention, and separate calls to the backbone per modality. In contrast, DSM allows for batching and accelerated inference, using only standard attention, with all modalities fused to limit the number of steps in the backbone decoder. In the context of this paper, this allows DSM to be trained for state-of-the-art ASR or TTS (see Figure 1), as shown in Section 4, with its performance being even competitive with offline approaches.

Multimodal language models. Transformer-based autoregressive models are the current main approach to sequence-to-sequence problems. They were introduced by Vaswani et al. (2017) for machine translation, and were soon extended to multimodal tasks, such as ASR (Radford et al., 2023) or visual understanding (Alayrac et al., 2022), by designing modality-specific encoders. More recently, neural codecs have provided compact, discrete representations of images (Esser et al., 2020) and audio (Zeghidour et al., 2022) that remove the need for modality-specific encoders inside the generative model, while providing a symmetrical processing of inputs and outputs which allows performing bidirectional tasks (e.g. speech-to-text and text-to-speech (Rubenstein et al., 2023)) with a single architecture. Défossez et al. (2024) introduce a multistream decoder architecture for spoken

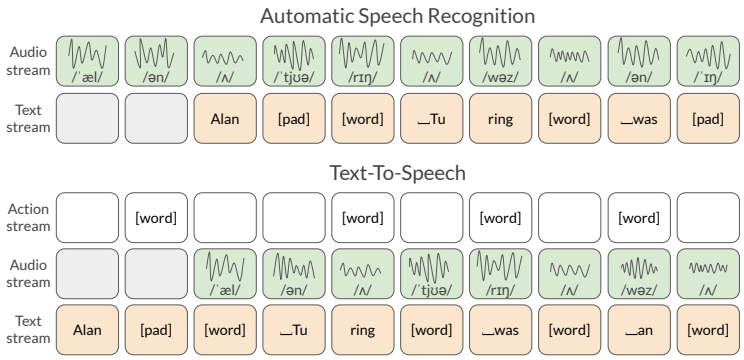

Figure 1: **Delayed streams modeling for speech-text tasks**. Depending on which stream is delayed with respect to the other, we solve either an ASR or a TTS task. For TTS, we further need an action stream for the model to let us know when it is ready to receive a new word.

dialogue, which predicts text and audio tokens in a streaming fashion, later applied by Labiausse et al. (2025) to real-time speech translation. In this work we extend the approach of Défossez et al. (2024), in order to reach state-of-the-art performance on the two most competitive speech-text tasks, namely ASR and TTS. Moreover, while Défossez et al. (2024) and Labiausse et al. (2025) operate with a delay specified before training, we propose delay conditioning for inference-time latency control without retraining. Our TTS covers both monologue and controllable dialog generation, a topic that was studied by CoVoMix (Zhang et al., 2024), although at a lower sample rate (8 kHz) and not streaming.

3 Method

Notation. We wish to solve a sequence-to-sequence task between two domains $\mathcal{X}$ and $\mathcal{Y}$. Each domain consists of sequences of vectors of all possible lengths, e.g.

$$\mathcal{X} = \bigcup_{T \in \mathbb{N}} \{(X_t) \in \mathbb{R}^{T \times d}\}, \qquad \mathcal{Y} = \bigcup_{T' \in \mathbb{N}} \{(Y_{t'}) \in \mathbb{R}^{T' \times d'}\}. \tag{1}$$

In the case where either X_t or Y_t is discrete-valued, we can use a one-hot representation for it in Eq. (1). We assume that we are given a joint probability distribution over the outer product domain $\mathcal{X} \times \mathcal{Y}$, and that we have the random variables $X \in \mathcal{X}$ and $Y \in \mathcal{Y}$, along with the joint distribution

$$\mathbb{P}[X, Y] = p(X, Y). \tag{2}$$

We also introduce $T \in \mathbb{N}$ (resp. T') the random variable indicating the length of X (resp. Y), along with the marginals $p(X)$ and $p(Y)$. For any sequence Z, and index t, we denote $Z_{<t} = (Z_1, \ldots, Z_{t-1})$, potentially empty if $t \leq 0$. We similarly define $Z_{\leq t}$, $Z_{\geq t}$, and $Z_{>t}$.

Sequence-to-sequence as joint modeling. Let's assume for this paragraph that $\mathcal{X}$ is the set of all possible monophonic waveforms sampled at 24 kHz, and $\mathcal{Y}$ is made of sequences of one-hot encoded vectors over a set of words. Intuitively, we assume there exists a coupling $p(X, Y)$ such that $p(X, Y)$ is high if Y represents the transcription of X, or conversely, if X represents a speech utterance of the text given by Y. Formally, the task of ASR corresponds to sampling from the distribution $\mathbb{P}[Y|X]$, while the task of TTS corresponds to sampling from the distribution $\mathbb{P}[X|Y]$. Thus, each task can be solved by accurately estimating both probability distributions,

$$q(X, Y) \approx \mathbb{P}[Y|X], \qquad q'(Y, X) \approx \mathbb{P}[X|Y]. \tag{3}$$

For simplicity, we now only focus on estimating $\mathbb{P}[Y|X]$, the inverse task being obtained by exchanging the definition of X and Y. We thus call $\mathcal{X}$ the input domain, and $\mathcal{Y}$ the output domain.

Auto-regressive modeling of Y. A good candidate for estimating $\mathbb{P}[Y|X]$ is auto-regressive modeling, with a Transformer model (Vaswani et al., 2017), under the extra assumption that the output domain $\mathcal{Y}$ can be discretized. Thus, one would estimate

$$q(y|X, Y_{<t}) \approx \mathbb{P}[Y_t = y|X, Y_{<t}]. \tag{4}$$

One can then sample Y auto-regressively, knowing X. Due to the lack of explicit structure between the time grid t of X and t' of Y, one would usually condition on the entirety of X, e.g. when using Transformer based models, either by prefixing the entire sequence X before the generation Y, or by providing X through cross-attention layers, which is mathematically equivalent. This forbids the use of the model in a streaming fashion, as the entire input signal X must be known ahead of time, and cannot be extended once the generation of Y has started. Such methods often require explicit and manual chunking and stitching operations, which also reduces their ability to be efficiently batched. Conversely, aligning X and Y to the same frame rate allows for batched streaming inference.

Aligning sequences for streaming prediction. We assume that both domains $\mathcal{X}$ and $\mathcal{Y}$ can share the same time grid, e.g. $(X_t) \in \mathbb{R}^{T \times d}$ and $(Y_t) \in \mathbb{R}^{T \times d'}$. We call two such aligned sequences *streams*. Then one can simply model

$$q_{\text{aligned}}\left(y | X_{\leq t}, Y_{<t}\right) \approx \mathbb{P}\left[Y_t = y | X_{\leq t}, Y_{<t}\right]. \tag{5}$$

Given $X \sim p(X)$, we sample auto-regressively from Eq. (5), *with a streaming context* X,

$$\tilde{Y}_1 \sim q_{\text{aligned}}\left(\tilde{Y}_1 | X_1\right), \qquad \tilde{Y}_t \sim q_{\text{aligned}}\left(\tilde{Y}_t | X_{\leq t}, \tilde{Y}_{<t}\right). \tag{6}$$

We would want that given $X \sim p(X)$, then $(X, \tilde{Y}) \sim (X, Y)$, so that in particular $\mathbb{P}\left[\tilde{Y} | X\right] \approx \mathbb{P}\left[Y | X\right]$. However this needs not be the case unless certain conditions are met.

The importance of causality. In particular, for $(X, \tilde{Y}) \sim (X, Y)$ to be true, $Y_{>t}$ must be independent of $X_{>t}$, knowing $X_{\leq t}$. To realize that, one can look at a simple counter-example taking $X_t \sim \mathcal{B}(0.5)$ independent Bernoulli variables, and $Y_t = X_t \oplus X_{t+1}$ the XOR of X_t and X_{t+1}. Clearly $\mathbb{P}\left[Y_t | X_{\leq t}, Y_{<t}\right] \sim \mathcal{B}(0.5)$ for all t, yet, given $X = (0, 1)$, one would have

$$Y_1 = 1 \quad \text{a.s.}, \qquad \tilde{Y}_1 \sim \mathcal{B}(0.5).$$

Thus $Y|X$ and $\tilde{Y}|X$ have different distributions. Intuitively, given that we do not sample X but teacher-force real-world data, we must ensure that when sampling $\tilde{Y}_t$, no future value of $X_{>t}$ might end up in "contradiction" with the value we sampled.

Delaying the output stream. In practice, this is achieved by delaying the output stream Y_t by a number of steps $\tau > 0$. Thus, we replace Eq. (5) by

$$q_\tau\left(y | X_{\leq t+\tau}, Y_{<t}\right) \approx \mathbb{P}\left[Y_t = y | X_{\leq t+\tau}, Y_{<t}\right], \tag{7}$$

and define $\tilde{Y}^\tau$, similarly to the procedure described in Eq. 6. Perfect independence is hard to achieve: in the case of ASR, a named entity might be ambiguous without context, and only future development in a discussion would resolve this ambiguity. Taking $\tau = T$ recovers the prefixing or cross-attention approaches presented earlier. In practice, there is a trade-off between the level of independence of Y_t with $X_{>t+\tau}$, and the latency of the method.

Architecture. DSM, depicted in Figure 2, contains three components: (i) an auto-regressive backbone, (ii) an input embedder for X and Y into the backbone, and (iii) a sampler for $\tilde{Y}^\tau$ conditioned on the output of the backbone. The backbone can be a Transformer architecture, optionally equipped with cross-attention layers to provide further non-streaming contextual information. The embedder for X and Y can be learnt embedding tables in the case where both domains are discrete. The embeddings are summed before going into the backbone. On the output side, we mask the loss on the tokens of X and only compute cross-entropy on Y. Finally, the conditional sampler can be a linear layer applied to the output of the backbone to derive logits if Y is discrete. It could also be a flow or diffusion model conditioned on the output of the backbone for the continuous case.

3.1 Representations of the speech and text domains

We demonstrate the DSM framework on ASR and TTS, where the two domains are text and audio.

Audio. Given a waveform $w \in \mathbb{R}^{d_s \cdot f_s}$ with the duration in seconds d_s and the sample rate $f_s = 24\,\text{kHz}$, we turn it into a more compact latent space using the Mimi codec (Défossez et al., 2024), giving us a sequence of tensors $Z^{\text{audio}} \in \mathbb{R}^{d_s \cdot f_r \times d_{\text{audio}}}$, with a frame rate of $f_r = 12.5\,\text{Hz}$. This latent space is discretized with Residual Vector Quantization (Zeghidour et al., 2022) (RVQ), giving us

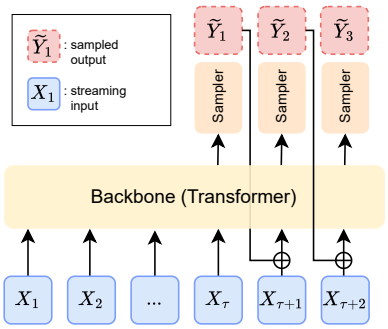

Figure 2: **DSM Architecture**. Transformer is fed with the streaming input X_t. After a delay τ, a sampler is fed with the output of the backbone samples $\tilde{Y}_t$. At the next step, the backbone receives both the sampled value and next streaming input, whose embeddings are summed.

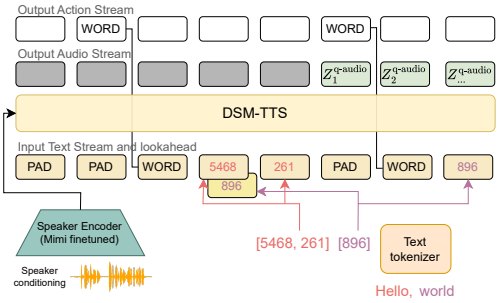

Figure 3: **DSM-TTS inference**. The input words "Hello, world" are tokenized. Until the model action stream outputs a `WORD`, it is fed with `PAD`. Then the first word's tokens are fed, including a look-ahead text stream. Once a delay τ (here 5 for illustration purposes) has accumulated, the model also outputs the audio.

a set of $Q \in [\![1, 32]\!]$ coarse-to-fine discrete values per time step with cardinality N_a=2048, each coming from one codebook in the RVQ, giving a quantized representation $Z^{\text{q-audio}} \in \{1, \dots, N_a\}^{d_s \cdot f_r \times Q}$.

Text. We tokenize text using a vocabulary of N_t, specifically trained on speech data transcriptions. Two tokens have a special meaning: `PAD` (indicating the absence of words at this time) and `WORD` (indicating the start of a new word following Défossez et al. (2024). Given a transcript, with word-level timestamps, of a waveform of duration d_s, its aligned text representation is $Z^{\text{text}} \in \{1, \dots N_t\}^{d_s \cdot f_r}$. For each word in the transcript represented by tokens $(x_1, \dots, x_n) \in \{1, \dots, N_t\}^n$ and starting at $s \in \mathbb{R}^+$ seconds, we define its start index $i = \text{floor}(s \cdot f_r)$, and store it as $Z^{\text{text}}_i \leftarrow$ `WORD`, $Z^{\text{text}}_{i+1} \leftarrow x_1$, $Z^{\text{text}}_{i+2} \leftarrow x_2$, etc. Any step in Z^{text} not assigned by a word token is given the special value `PAD`.

3.2 DSM for automatic speech recognition: DSM-ASR

For ASR, we consider $X = Z^{\text{q-audio}}$ and $Y = Z^{\text{text}}$. By predicting the word tokens of Y, we learn to transcribe audio, while computing the loss on `PAD` and `WORD` tokens trains the model to predict the precise boundaries of each word. At inference time, we teacher-force the audio tokens of X and sample the full sequence Z^{text} to obtain a transcription along with timestamps with a precision of 80ms (frame size). This is allowed by the fact that we apply a constant delay to all words in the sequence, meaning we only need to shift the output timestamps back by the same value to recover the true timestamps.

Deriving aligned speech-text data. We are looking for fine-grained alignment between speech and text, however speech datasets are typically aligned at the level of the sentence (Panayotov et al., 2015). Conveniently, `whisper-timestamped` (Louradour, 2023) provides automatic transcriptions with word-level timestamps. We rely on these pseudo-labels for the pretraining phase of DSM-ASR. We then finetune on a mixture of public datasets with ground-truth transcripts (see details in Section 4.2), which pose two challenges. First, the automatic transcriptions extracted by Whisper in pretraining are formatted with capitalization and punctuation, but the level of formatting varies a lot between datasets. To address this, we train a 300M prefix-LM for automatic formatting, on a dataset of formatted Whisper transcripts. A second challenge is that these ground-truth transcripts do not have word-level alignment. We derive those by producing pseudo-transcripts with Whisper, and reconciling them with the formatted transcript using a Dynamic Time Warping algorithm (Giorgino, 2009).

Delay conditioning for inference-time control. As shown in Section 4.3.1, transcription quality is heavily dependent on the delay between audio and text. Thus, training DSM-ASR with a fixed delay requires choosing a latency/quality trade-off beforehand, and retraining a new model for each delay, despite the training task remaining fundamentally the same. To instead control this trade-off at inference, we train DSM-ASR over random delays, sampled for each sequence. The model is additionally conditioned on a cosine embedding (Vaswani et al., 2017) of the delay (expressed in milliseconds), added to the inputs. Experiments in Section 4.3.1 compare this model to the models trained with a fixed delay and show that the effective delay precisely respects the conditioning value.

3.3 DSM for text-to-speech

We further apply DSM to TTS, taking $X = Z^{\text{text}}$, $Y = Z^{\text{q-audio}}$. We use a stream delay of 1.28s (or 16 steps) on the output audio. For sampling along the Q dimension in $Z^{\text{q-audio}}$, we use a RQ-Transformer as a sampler (Lee et al., 2022; Défossez et al., 2024), i.e. a smaller Transformer conditioned on the output of the backbone at each timestep and performing autoregressive modeling along the Q dimension. All the backbone inputs (generated audio tokens and next word token input) are fed through learnt embeddings and summed. We are confronted with the problem that the input domain is no longer plain text, but text properly padded for time alignment. While at train time we can teacher-force the ground-truth padded text, this is not the case for a novel text to synthesize at inference time.

Action output stream. We add an extra stream to the TTS outputs, whose goal is to predict whether the next input text token will be a `WORD` token or not. This special input token indicates that a new word is starting, and that its tokens are going to follow as inputs. This extra stream controls an inference-time *action*: when predicted by the model, we will feed as input the text tokens for the next word over the next time steps. While these are being fed, the model is not allowed to output another `WORD` action. The action output stream is not fed back into the model as it is redundant with the text stream input.

Lookahead second text stream. The action stream allows the model to predict the next word position, although the model has no knowledge of its content for making that decision. The delay between text and audio only provides context for the audio generation, however, the decision on where to insert pauses and words has no such context. Given a sequence of words $m_1, m_2, \ldots$, the lookahead text stream feeds the tokens of the words m_{i+l} to the backbone while the primary text feed contains the tokens of words m_i.

Speaker conditioning. We provide speaker embeddings for up to 5 speakers. Each speaker is represented by a 10s audio extract of the same speaker outside of the training segment. Speakers are identified using the diarization tool Pyannote (Bredin, 2023) in the training data. If more than 5 speakers are present in the segment, only 5 at random are kept for the speaker embeddings. If less than 5 speakers are present, the remaining speaker slots are filled with learnt padding values. Each speaker audio extract is encoded with a *speaker encoder* and results in a speaker embedding with a fixed dimension. We concatenate the speaker embedding from the different speakers, sum them with an absolute positional embedding, and feed them through cross-attention layers to the backbone. The speaker encoder has the same architecture as the encoder of the Mimi codec, and is initialized with its weights. We keep the weights of the convolutional layers frozen for stability, but let its Transformer layers be fine-tuned in an end-to-end fashion with the language model conditioned on it.

Change of turn tokens. We indicate change of turns between the first speaker in the speaker embedding, called the *main speaker*, and any other speaker. When the main speaker starts talking, their first word is prefixed with a special `MAIN` token in the text stream. When another speaker starts speaking after the main speaker, a special `OTHER` token is inserted. At inference time, we can thus make controllable dialogs by feeding the model with speaker embeddings for the two speakers, and controlling the change of turn by inserting the `MAIN` and `OTHER` special tokens.

Classifier free guidance. We use classifier free guidance (CFG) (Ho & Salimans, 2022; Kreuk et al., 2023), both with respect to the speaker conditioning, and also with respect to the text, that is, we replace at inference time the logits for a given timestep t and codebook index q, given $\alpha \geq 1$, with

$$l_{t,q} = l^{\emptyset}_{t,q} + \alpha \left(l^{\text{text,speaker}}_{t,q} - l^{\emptyset}_{t,q} \right), \tag{8}$$

where $l^{\emptyset}_{t,q}$ are the logit estimates obtained by feeding no text, action or lookahead inputs to the model, and no speaker embedding, and $l^{\text{text,speaker}}_{t,q}$ are the conditioned logits estimates. No CFG is applied on the action stream logits. The model is trained with an independent dropout of 20% on the speaker embedding and on the input text. Unless stated otherwise, we use $\alpha = 2$.

An overview of the whole DSM-TTS inference process is shown in Figure 3.

Table 1: **Short-form ASR performance**. We report Word Error Rates (WER, %) for DSM-ASR and selected non-streaming baselines from the OpenASR leaderboard, along with streaming baselines.

Model	Avg.	AMI	Earnings22	Gigaspeech	LS Clean	LS Other	SPGISpeech	TED-LIUM	Voxpopuli
Non-streaming									
Whisper medium.en	8.1	16.7	12.6	11.0	3.0	5.9	3.3	4.1	9.6
Whisper Large-v3	7.5	16.0	11.3	10.0	2.0	3.9	2.9	3.9	9.5
Voxtral Mini	7.1	16.3	10.7	10.2	1.9	4.1	2.4	3.7	7.1
ElevenLabs Scribe	6.9	14.4	12.1	9.7	1.8	3.3	3.3	3.2	7.2
CrisperWhisper	6.7	**8.7**	12.9	10.2	1.8	4.0	2.7	3.2	9.8
Canary-Flash	6.4	13.1	12.8	9.9	**1.5**	**2.9**	2.0	3.1	**5.6**
Phi-4 Multimodal	6.1	11.5	**10.5**	9.8	1.7	3.8	3.1	2.9	5.9
Parakeet-TDT-v2	6.1	11.2	11.2	9.7	1.7	3.2	2.2	3.4	6.0
Canary-Qwen-2.5B	**5.6**	10.2	**10.5**	**9.4**	1.6	3.1	**1.9**	**2.7**	5.7
Streaming									
Whisper medium.en	9.0	22.1	13.4	10.4	3.0	6.2	3.7	4.7	8.6
Whisper large-v3	9.4	18.4	11.0	10.0	8.4	12.6	3.2	3.8	7.9
SeamlessStreaming	19.7	45.0	31.8	21.6	6.8	10.6	15.4	12.4	13.9
Parakeet-TDT-v2	7.0	**11.9**	11.8	10.2	3.3	4.7	3.4	3.9	**6.6**
DSM-ASR	**6.4**	12.2	**11.0**	**9.8**	**1.7**	**4.3**	**2.0**	**3.4**	6.8

4 Experiments

4.1 Architectural hyperparameters

We use a Transformer (Vaswani et al., 2017) backbone with RoPE positional encoding (Su et al., 2024). For the DSM-TTS experiments, we use a 1B parameters backbone with a 2048 dimension latent space, GLU feed-forward units, 16 layers, and 16 heads of attention. The TTS model also receives the speaker embedding through cross-attention layers. The sampler is a Transformer along the codebook Q dimension described in Section 3.1, with no context over the time axis, with a dimension of 1024, 4 layers for each codebook, with a linear layer to estimate the logits. While Défossez et al. (2024) used a different set of parameters for each codebook in this Transformer over Q, we follow Labiausse et al. (2025), using only individual set of parameters for the first 8 codebooks, then sharing parameters per group of 8 codebooks, for a total of 1.8B parameters with the backbone. The text tokenizer is trained on bilingual French/English data, with a cardinality $N_t^{\text{tts}} = 8000$. The model uses a delay τ of 1.6s, or 16 steps, and a lookahead stream with $l = 2$ (Section 3.3).

The DSM-ASR uses a 2.6B parameters backbone, with 2048 dimensions, 48 layers, and 32 attention heads, and a linear to predict the logits over the text vocabulary, with a cardinality $N_t^{\text{asr}} = 4000$, trained for English only. We experiment with two flavors of the ASR model: one uses a single fixed delay value for all examples, the other has a variable delay τ which is sampled per batch item in a range of $[0.25, 4]$s (Section 3.2).

4.2 Training protocol

Pretraining. We use an audio collection of 2.5 million hours of publicly available audio content in English and French transcribed with `whisper-timestamped`. Given the synthetic nature of text transcripts, this phase amounts to hard distillation of `whisper-timestamped`. We train DSM-ASR on random 90s segments for 1.6M steps, on 48 H100s. DSM-TTS is trained on 150s audio extracts, on 32 H100s, 750k updates with batch size 64. We use AdamW (Loshchilov & Hutter, 2019), a cosine learning rate schedule with linear warmup, with an initial rate of $2 \cdot 10^{-4}$ for the TTS, and $4 \cdot 10^{-4}$ for the ASR, and a weight decay of 0.1.

Finetuning (DSM-ASR). We then finetune the model on a collection of public datasets with ground-truth transcripts, described in Appendix A.1 and totaling 28k hours. This training stage lasts for 100k updates with batches of 128 examples, using 16 H100s. We augment training by using the codebook dropout (Défossez et al., 2024). We then adapt the model to long-form inputs, which most public datasets lack, by constructing a long-form mixture described in Appendix A.2. We run this stage for DSM-ASR for 25k updates with batch size 32, using 16 H100s.

Finetuning (DSM-TTS). For safety reasons, we want this open-source model to be usable only with proper speaker embeddings from our speaker encoder, which we keep closed-source. This is not compatible with the CFG formula given by Eq. (8), which requires the model to be usable unconditionally. Thus, we fine-tune the TTS with distillation of the CFG step, using an extra tensor

Table 2: **Long-form ASR performance**. We report Word Error Rates (WER, %) across four long-form datasets for DSM-ASR and a set of streaming and non-streaming baselines.

MODEL	AVG.	TED-LIUM	MEANWHILE	REV16	EARNINGS21
Non-streaming					
DISTIL-LARGE V2	8.7	3.7	7.8	12.2	11.2
WHISPER-LARGE-V2	9.0	4.4	6.3	13.6	11.8
Streaming					
WHISPER MEDIUM.EN	9.0	3.9	6.7	13.0	12.5
WHISPER LARGE-V3	8.1	3.4	6.1	11.4	11.4
PARAKEET-0.6B-TDT-V2	**7.8**	3.7	**5.0**	**11.0**	11.3
DSM-ASR	7.9	**2.9**	5.7	12.3	**10.6**

conditioning summed with each backbone input to indicate value for $\alpha \in \{1, 1.5, 2, 2.5, 3, 3.5, 4\}$. The model is fined tuned for 2400 updates with a learning rate of $1 \cdot 10^{-6}$.

4.3 Automatic Speech Recognition

We evaluate DSM-ASR (with a default fixed delay of 2.5s) in terms of transcription quality, latency, and timestamps precision. We consider short-form transcription (shorter than 30s), as it is the focus of the OpenASR Leaderboard (Srivastav et al., 2023). We also look at streaming inference for long-form transcription (up to 2 hours).

Baselines. We benchmark DSM-ASR against leading models of the OpenASR Leaderboard, including Whisper (Radford et al., 2023), Canary-Flash (Zelasko et al., 2025), Phi-4 Multimodal Instruct (Abouelenin et al., 2025), Parakeet (Xu et al., 2023), Voxtral Mini (Liu et al., 2025) and as well as the closed-source ElevenLabs API [1]. Notably, all these models perform non-streaming ASR, as they require access to the full input sequence. We thus include a streaming variant of Whisper (Macháček et al., 2023) (with a delay of 2.5s) and SeamlessStreaming (Barrault et al., 2023). We also include a (block) streaming version of Parakeet-TDT-v2, evaluated via running the official scripts with a 2.5s delay that matches the delay used by DSM-ASR.[2] For long-form transcription, we add the Distil-Whisper (Gandhi et al., 2023) variant.

Transcription quality. We report micro-averaged Word Error Rate (WER), which avoids over-emphasizing short sequences, and is the standard computation used in the OpenASR Leaderboard, of which we use the official evaluation codebase. Throughout this section, we use the Whisper normalizer for English (Radford et al., 2023).

Latency. We evaluate the latency of streaming models as the average delay between the real timestamp of a word, and the time when this word is transcribed in the output. In the absence of ground-truth timestamps, we use pseudo-timestamps on Librispeech test-clean (Panayotov et al., 2015) provided by Lugosch et al. (2019). These timestamps were obtained by Montreal Forced Aligner (McAuliffe et al., 2025), and use them as reference.

Timestamps. See Appendix E for a complete description.

4.3.1 ASR Results

Short-form transcription. Table 1 shows that DSM-ASR is significantly better than streaming baselines, and even competitive with the best, non-streaming models of the OpenASR Leaderboard. With an average WER of 6.4%—with 6.1% being the current best score of the leaderboard—DSM-ASR is remarkably the only streaming model among top ASR systems. In Appendix E, we see that, in terms of timestamp precision, DSM-ASR performs significantly better than Whisper Large-v3 while somewhat underperforming CrisperWhisper, though with a better WER.

Long-form transcription. Table 2 reports WER values across 4 long-form datasets with sequences up to 2 hours: TED-LIUM (Hernandez et al., 2018), Meanwhile (Radford et al., 2023), Rev16 (Radford et al., 2023), and Earnings21 (Rio et al., 2021). The non-streamed version of Parakeet runs out of memory on a 80Gb GPU even when we use single-example batches, so we exclude it from the analysis.

[1] elevenlabs.io.

[2] We use chunk size of 10 frames (0.8s) and right context of 21 frames (1.7s).

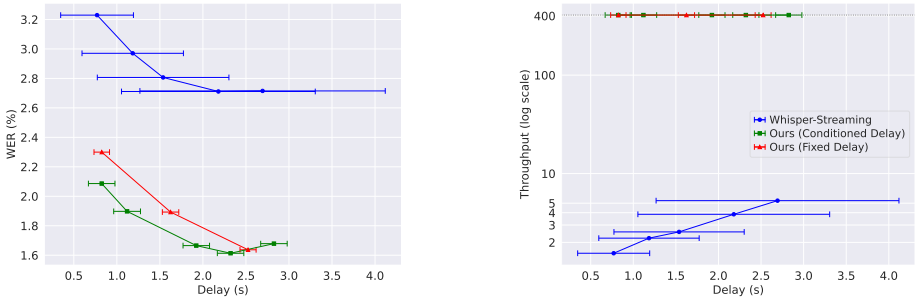

Figure 4: **ASR WER and throughput vs. delay**. Word Error Rate (WER, %) (left) and throughput (right) in function of the delay. Throughput is the product between Real-Time Factor and batch size.

Table 3: **Long-form TTS WER**. We compare open-source and closed-source baselines. Short-form inference is ran with a short context. ElevenLabs is evaluated on a limited subset due to its cost.

Model	WER English (%) (↓)			WER French (%) (↓)		
	Avg.	Dialogs	Monologues	Avg.	Dialogs	Monologues
Short-form inference						
Orpheus	3.51	3.91	3.11	-	-	-
CSM	3.82	2.95	4.68	-	-	-
Dia	2.79	2.40	3.18	14.20	12.49	15.91
Chatterbox	1.95	1.43	2.47	-	-	-
DSM-TTS (ours, turn-by-turn)	2.06	1.36	2.75	3.20	2.74	3.66
Long-form inference						
DSM-TTS (ours)	**1.72**	**1.15**	**2.29**	**2.96**	**2.66**	**3.26**
Long-form inference, small subset						
DSM-TTS (ours)	2.35	1.76	**2.94**	**2.78**	**2.91**	**2.65**
ElevenLabs Flash	2.59	1.13	4.05	4.51	4.38	4.64
ElevenLabs Multilingual v2	**2.01**	**0.91**	3.10	2.93	3.12	2.75

From Table 2, we see that DSM-ASR outperforms streaming and non-streaming baselines except for Parakeet, which it closely matches (7.8 vs 7.9).

Distillation. At the pretraining stage, DSM-ASR is trained using pseudo-transcripts produced by `whisper-timestamped` (Louradour, 2023), which wraps Whisper Medium. As we can see from Table 1, the teacher model has the average WER of 8.1, while DSM-ASR gets 6.4. Without finetuning on datasets with ground-truth transcripts, DSM-ASR gets WER of 7.1.

We hypothesize that the observed improvements over the teacher come from (a) smoothing across a larger and more diverse set of real-world audio, implicitly leading to a domain adaptation, (b) low-temperature sampling that would eliminate non-systematic errors of the teacher model. Note that such improvements through teacher/student distillation have been observed before, e.g. on ImageNet classification (Yalniz et al., 2019). We also perform augmentations such as codebook dropout. After the fine-tuning stage, DSM-ASR gets WER of 6.4%. Here, we train on ground-truth transcripts that come with the standard ASR datasets. At this stage, Whisper is only used to derive the timestamps.

Delay conditioning and latency. Figure 4 (left) compares the WER obtained for DSM-ASR with and without delay conditioning, along with Whisper-Streaming. We observe that the delay of Whisper-Streaming has a large variance, while DSM-ASR has a precision of ∼300ms around its target delay. Interestingly, training a single DSM-ASR model with delay conditioning outperforms fixed delay variants. Figure 4 (right) shows the throughput on an H100 GPU: DSM-ASR can process 400 sequences simultaneously while being real-time and its throughput is independent of the delay. This is unlike Whisper-Streaming which reduces its delay by re-evaluating the partial input sequence more frequently, increasing the computational cost. Combined with the fact that Whisper-Streaming does not allow for batching, this results in a 100x lower throughput than that of DSM-ASR.

4.4 Text-To-Speech experiments

Evaluation datasets. We collect a novel dataset for long-form TTS evaluation in English and French. We first use news articles from the NTREX-128 (Federmann et al., 2022) text dataset, given 123

Table 4: **Long-form TTS Speaker Similarity**. We compare open-source and closed-source baselines. Short-form inference is ran with a short context. ElevenLabs runs on a limited subset due to its cost.

Model	Speaker Sim. English (%) (↑)			Speaker Sim. French (%) (↑)		
	Avg.	Dialogs	monologues	Avg.	Dialogs	monologues
Short-form inference						
orpheus	38.86	36.80	40.92	-	-	-
csm	74.44	65.83	83.05	-	-	-
dia	62.48	57.03	67.93	54.99	50.13	59.85
chatterbox	66.92	63.32	70.51	-	-	-
DSM-TTS (ours, turn-by-turn)	**80.90**	**77.67**	**84.13**	**80.46**	**76.63**	**84.29**
Long-form inference						
DSM-TTS (ours)	76.46	74.06	78.87	75.96	73.21	78.72
Long-form inference, small subset						
DSM-TTS (ours)	**76.40**	**73.43**	**79.37**	**75.92**	**72.98**	**78.86**
ElevenLabs Flash	46.84	45.39	48.30	53.55	53.97	53.13
ElevenLabs Multilingual v2	56.40	55.36	57.44	62.10	60.40	63.79

Table 5: **Subjective evaluations on TTS**. We compare with baselines over two axes through human evaluations: speech quality, measured as MUSHRA scores (1–100, along with std. of the mean), and speaker-similarity win-rates, summarized as ELO scores (with 95% confidence intervals).

Model	English		French	
	Quality (↑)	Spk. Sim. (↑)	Quality(↑)	Spk. Sim.(↑)
Orpheus	27.1 ± 2.1	1716.3 ± 35.6	-	-
CSM	39.6 ± 2.2	1997.6 ± 19.4	-	-
Dia	43.6 ± 2.4	1926.9 ± 21.3	24.9 ± 2.2	1679.0 ± 41.7
Chatterbox	$\mathbf{67.7} \pm 2.0$	2068.3 ± 18.7	-	-
DSM-TTS (ours)	54.8 ± 2.3	2048.3 ± 18.6	62.5 ± 2.0	2098.5 ± 21.1
DSM-TTS (ours, turn-by-turn)	53.1 ± 2.2	$\mathbf{2092.6} \pm 18.3$	61.9 ± 2.0	$\mathbf{2179.1} \pm 22.4$
ElevenLabs Flash	65.0 ± 2.2	1932.3 ± 20.9	61.5 ± 2.3	1894.3 ± 20.4
ElevenLabs Multilingual v2	59.7 ± 2.4	2035.3 ± 19.2	$\mathbf{66.4} \pm 2.0$	2047.3 ± 21.1

monologues per language. To evaluate controllable dialog capabilities, we use 110 synthetic scripts per language generated by an LLM, spanning three categories: daily life, technical, and number-heavy discussions. For voice conditioning, we use samples from the test set of VCTK (Yamagishi et al., 2019) for English, and from the test and valid sets of CML (Oliveira et al., 2023) for French. We provide examples and more details in the Appendix D, while the dataset and evaluation code is available at github.com/kyutai-labs/tts_longeval.

Metrics. We evaluate the per-document WER, using text normalization from Whisper (Radford et al., 2023). We collect subjective metrics covering both the speaker similarity to the conditioning and overall speech quality, see Appendix A for more details.

Baselines. We compare to open-source models Chatterbox, Dia, Orpheus, and CSM, as well as ElevenLabs.[3] Dia and ElevenLabs support French and English, while Chatterbox, Orpheus and CSM only support English[4]. Chatterbox, Dia, Orpheus and CSM can be speaker-conditioned through prefixing, with Dia and CSM supporting dialogs. For Chatterbox, Orpheus and ElevenLabs, dialogs are emulated by concatenating single-speaker turns. Details of how baselines are evaluated are provided in Appendix G.

4.4.1 TTS Results

Main results. As seen in Table 3, our approach provides the lowest WER across all languages for both monologues and dialogs. Our method is the only one to run long-form inference across all cases, CSM showing strong degradation when running with longer sequences, Dia only being trained for 20s output, and ElevenLabs requiring per-turn generation for dialogs. We also provide speaker similarity measurements in Table 4. We report subjective results in Table 5. When evaluating turn-by-turn (e.g. short-form like the other baselines), we outperform all existing methods in terms of speaker similarity, while still surpassing commercial methods when using long-form generation. In terms of quality, Chatterbox is rated the highest, with DSM-TTS surpassing the other open source baselines.

[3] resemble-ai/chatterbox, nari-labs/dia, canopyai/Orpheus-TTS, SesameAILabs/csm and elevenlabs.io.

[4] A new version of Chatterbox supports multiple languages but was not available at the time of this study.

Table 6: **TTS: Latency and throughput**. We compare the inference performance of DSM-TTS and selected baselines. Real-Time Factor (RTF) is higher than 1 if the model can produce audio in real time. Throughput is the product between Real-Time-factor and batch size.

MODEL	MODEL SIZE	LATENCY (MS)(↓)	RTF(↑)	THROUGHPUT(↑)
DIA	1.6B	-	0.7	0.7
CSM	1.5B	-	1.0	1.0
ORPHEUS	3.8B	-	0.7	0.7
CHATTERBOX	0.8B	-	1.8	1.8
DSM-TTS B.S.=1	1.8B	150	3.2	3.2
DSM-TTS B.S.=32	1.8B	380	2.4	76.8
DSM-TTS B.S.=64	1.8B	403	2.1	137.3

Table 7: **Results of DSM-TTS trained on public datasets, evaluated on LibriSpeech test-clean**. We compare to F5-TTS (Chen et al., 2024b), following their evaluation framework. RTF and Throughput were computed on a H100 for both methods.

Model	#Param.	#Data	WER (%) ↓	SIM-o ↑	Latency ↑	RTF ↑	Throughput ↑
F5-TTS (16 flow steps)	336M	100K (EN, ZH)	2.53	0.66	~ 800 ms	**14.2**	14.2
F5-TTS (32 flow steps)			2.42	0.66	~ 400 ms	7.1	7.1
DSM-TTS (ours, nq=16, b.s.=1)	750M	88K (EN)	1.95	0.67	**139ms**	4.4	4.4
DSM-TTS (ours, nq=32, b.s.=1)	900M	88K (EN)	**1.68**	**0.71**	172ms	2.7	2.7
DSM-TTS (ours, nq=32, b.s.=32)	900M	88K (EN)	**1.68**	**0.71**	351ms	2.3	**74.2**

Note that we kept all methods with their original sample rate (e.g. 44.1kHz for ElevenLabs) which can contribute to the difference.

Throughput and latency. Our method is easily batchable, leading to gains in throughput while staying compatible with real-time generation. As shown in Table 6, on a single H100 the amount of audio generated is 100x real-time. More details are provided in Appendix I.

Training on public data only. For reproducibility, we provide results training from scratch DSM-TTS with the public training data described in Section J, totaling 88k hours of speech. We train a 300M parameters backbone model with either 16 or 32 codebooks. We evaluate on Librispeech test clean (Panayotov et al., 2015) and on Seed test en (Anastassiou et al., 2024). We provide the results in Tables 7 and 8, with comparisons to F5-TTS (Chen et al., 2024b) and CosyVoice3-1.5B (RL) (Du et al., 2025). Refer to Appendix J for details.

Further results and ablations. Finally, we provide ablations on the codec choice, and the contribution from the action and lookahead stream in the Appendix L.

Watermarking. A number of open source baselines in Table 3 rely on watermarking to protect against negative use of their models. Note that for open source models, this step can be disabled. Besides, we noticed that a single round of encoding and decoding with the Mimi codec with 32 codebooks completely or largely remove the watermarks commonly used, see Appendix K. This suggests that securing the use and traceability of modern TTS models is still an open question.

DSM-ASR and DSM-TTS as a speech interface for LLMs. We combine DSM-ASR, DSM-TTS, and Gemma 3 (Gemma Team et al., 2025) into an LLM voice chat application with sub-second latency. The application is available at `https://unmute.sh`.

5 Conclusion

We introduce Delayed Streams Modeling, a flexible framework for streaming sequence-to-sequence learning. DSM provides a remarkable trade-off between quality and latency, and an unprecedented throughput among streaming models. Focusing on speech-text tasks, DSM-ASR is the first streaming ASR model to provide timestamped, formatted transcripts that competes with the top offline models, while DSM-TTS is competitive with non-streaming baselines while being the only model providing long form synthesis. In future work, we will extend DSM to more sequential multimodal tasks. In particular, one limitation of our approach is the need for aligned domains, which reduces the amount of gold-standard ground-truth data that can be used for training.

Societal impact. We acknowledge that streaming naturalistic speech with voice conditioning opens up both opportunities in inclusive human-machine interactions and risks of fraudulent impersonation. Ad-

Table 8: **Results of DSM-TTS trained on public datasets, evaluated on Seed test en**. See Section J for details.

Model	**WER** (%) ↓	**SIM-o** ↑
F5-TTS (Chen et al., 2024b)	1.83	**0.71**
Cosyvoice 3-1.5B (RL) (Du et al., 2025)	**1.45**	0.70
DSM-TTS (ours, nq=16)	1.58	0.70
DSM-TTS (ours, nq=32)	1.71	**0.71**

dressing the latter requires that public access to such technologies is accompanied by proper user terms, voice verification mechanisms , and resilient watermarking of generated content. Given the limitations of such existing approaches, in particular for open source models, we have not open sourced the voice conditioning module for our best TTS model, only providing pre-computed speaker embeddings.

References

Abouelenin, A., Ashfaq, A., Atkinson, A., Awadalla, H., Bach, N., Bao, J., Benhaim, A., Cai, M., Chaudhary, V., Chen, C., Chen, D., Chen, D., Chen, J., Chen, W., Chen, Y., Chen, Y., Dai, Q., Dai, X., Fan, R., Gao, M., Gao, M., Garg, A., Goswami, A., Hao, J., Hendy, A., Hu, Y., Jin, X., Khademi, M., Kim, D., Kim, Y. J., Lee, G., Li, J., Li, Y., Liang, C., Lin, X., Lin, Z., Liu, M., Liu, Y., Lopez, G., Luo, C., Madan, P., Mazalov, V., Mitra, A., Mousavi, A., Nguyen, A., Pan, J., Perez-Becker, D., Platin, J., Portet, T., Qiu, K., Ren, B., Ren, L., Roy, S., Shang, N., Shen, Y., Singhal, S., Som, S., Song, X., Sych, T., Vaddamanu, P., Wang, S., Wang, Y., Wang, Z., Wu, H., Xu, H., Xu, W., Yang, Y., Yang, Z., Yu, D., Zabir, I., Zhang, J., Zhang, L. L., Zhang, Y., and Zhou, X. Phi-4-mini technical report: Compact yet powerful multimodal language models via mixture-of-loras. *CoRR*, abs/2503.01743, 2025. doi: 10.48550/ARXIV.2503.01743.

Alayrac, J., Donahue, J., Luc, P., Miech, A., Barr, I., Hasson, Y., Lenc, K., Mensch, A., Millican, K., Reynolds, M., Ring, R., Rutherford, E., Cabi, S., Han, T., Gong, Z., Samangooei, S., Monteiro, M., Menick, J. L., Borgeaud, S., Brock, A., Nematzadeh, A., Sharifzadeh, S., Binkowski, M., Barreira, R., Vinyals, O., Zisserman, A., and Simonyan, K. Flamingo: a visual language model for few-shot learning. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A. (eds.), *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022.

Anastassiou, P., Chen, J., Chen, J., Chen, Y., Chen, Z., Chen, Z., Cong, J., Deng, L., Ding, C., Gao, L., et al. Seed-tts: A family of high-quality versatile speech generation models. *arXiv preprint arXiv:2406.02430*, 2024.

Bahdanau, D., Cho, K., and Bengio, Y. Neural machine translation by jointly learning to align and translate. In Bengio, Y. and LeCun, Y. (eds.), *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015.

Barrault, L., Chung, Y., Meglioli, M. C., Dale, D., Dong, N., Duppenthaler, M., Duquenne, P., Ellis, B., Elsahar, H., Haaheim, J., Hoffman, J., Hwang, M., Inaguma, H., Klaiber, C., Kulikov, I., Li, P., Licht, D., Maillard, J., Mavlyutov, R., Rakotoarison, A., Sadagopan, K. R., Ramakrishnan, A., Tran, T., Wenzek, G., Yang, Y., Ye, E., Evtimov, I., Fernandez, P., Gao, C., Hansanti, P., Kalbassi, E., Kallet, A., Kozhevnikov, A., Gonzalez, G. M., Roman, R. S., Touret, C., Wong, C., Wood, C., Yu, B., Andrews, P., Balioglu, C., Chen, P., Costa-jussà, M. R., Elbayad, M., Gong, H., Guzmán, F., Heffernan, K., Jain, S., Kao, J., Lee, A., Ma, X., Mourachko, A., Peloquin, B. N., Pino, J., Popuri, S., Ropers, C., Saleem, S., Schwenk, H., Sun, A. Y., Tomasello, P., Wang, C., Wang, J., Wang, S., and Williamson, M. Seamless: Multilingual expressive and streaming speech translation. *CoRR*, abs/2312.05187, 2023. doi: 10.48550/ARXIV.2312.05187.

Beyer, L., Steiner, A., Pinto, A. S., Kolesnikov, A., Wang, X., Salz, D., Neumann, M., Alabdulmohsin, I., Tschannen, M., Bugliarello, E., Unterthiner, T., Keysers, D., Koppula, S., Liu, F., Grycner, A., Gritsenko, A. A., Houlsby, N., Kumar, M., Rong, K., Eisenschlos, J., Kabra, R., Bauer, M., Bosnjak, M., Chen, X., Minderer, M., Voigtlaender, P., Bica, I., Balazevic, I., Puigcerver, J., Papalampidi, P., Hénaff, O. J., Xiong, X., Soricut, R., Harmsen, J., and Zhai, X. Paligemma: A versatile 3b VLM for transfer. *CoRR*, abs/2407.07726, 2024. doi: 10.48550/ARXIV.2407.07726.

Bradley, R. A. and Terry, M. E. Rank analysis of incomplete block designs: The method of paired comparisons. *Biometrika*, 39(3-4):324–345, 12 1952. ISSN 0006-3444. doi: 10.1093/biomet/39.3-4.324.

Bredin, H. pyannote.audio 2.1 speaker diarization pipeline: principle, benchmark, and recipe. In *Proc. INTERSPEECH 2023*, 2023.

Carletta, J. Unleashing the killer corpus: experiences in creating the multi-everything ami meeting corpus. *Language Resources and Evaluation*, 41:181–190, 2007.

Caron, F. and Doucet, A. Efficient bayesian inference for generalized bradley-terry models, 2010.

Chan, W., Jaitly, N., Le, Q., and Vinyals, O. Listen, attend and spell: A neural network for large vocabulary conversational speech recognition. In *2016 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, pp. 4960–4964. IEEE, 2016.

Chen, G., Chai, S., Wang, G., Du, J., Zhang, W.-Q., Weng, C., Su, D., Povey, D., Trmal, J., Zhang, J., et al. Gigaspeech: An evolving, multi-domain asr corpus with 10,000 hours of transcribed audio. *arXiv preprint arXiv:2106.06909*, 2021a.

Chen, J., Ma, M., Zheng, R., and Huang, L. Direct simultaneous speech-to-text translation assisted by synchronized streaming ASR. In Zong, C., Xia, F., Li, W., and Navigli, R. (eds.), *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pp. 4618–4624, Online, August 2021b. Association for Computational Linguistics. doi: 10.18653/v1/2021.findings-acl.406. URL https://aclanthology.org/2021.findings-acl.406/.

Chen, P., Sun, S., Shan, C., Yang, Q., and Xie, L. Streaming decoder-only automatic speech recognition with discrete speech units: A pilot study. *arXiv preprint arXiv:2406.18862*, 2024a.

Chen, Y., Niu, Z., Ma, Z., Deng, K., Wang, C., Zhao, J., Yu, K., and Chen, X. F5-tts: A fairytaler that fakes fluent and faithful speech with flow matching. *arXiv preprint arXiv:2410.06885*, 2024b.

Chiu, C. and Raffel, C. Monotonic chunkwise attention. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net, 2018.

Chiu, C., Kannan, A., Prabhavalkar, R., Chen, Z., Sainath, T. N., Wu, Y., Han, W., Zhang, Y., Pang, R., Kishchenko, S., Nguyen, P., Narayanan, A., Liao, H., and Zhang, S. A comparison of end-to-end models for long-form speech recognition. In *IEEE Automatic Speech Recognition and Understanding Workshop, ASRU 2019, Singapore, December 14-18, 2019*, pp. 889–896. IEEE, 2019. doi: 10.1109/ASRU46091.2019.9003854.

Défossez, A., Copet, J., Synnaeve, G., and Adi, Y. High fidelity neural audio compression. *Transactions on Machine Learning Research*, 2023.

Défossez, A., Mazaré, L., Orsini, M., Royer, A., Pérez, P., Jégou, H., Grave, E., and Zeghidour, N. Moshi: a speech-text foundation model for real-time dialogue. *CoRR*, abs/2410.00037, 2024. doi: 10.48550/ARXIV.2410.00037.

Del Rio, M., Ha, P., McNamara, Q., Miller, C., and Chandra, S. Earnings-22: A practical benchmark for accents in the wild. *arXiv preprint arXiv:2203.15591*, 2022.

Du, Z., Gao, C., Wang, Y., Yu, F., Zhao, T., Wang, H., Lv, X., Wang, H., Ni, C., Shi, X., et al. Cosyvoice 3: Towards in-the-wild speech generation via scaling-up and post-training. *arXiv preprint arXiv:2505.17589*, 2025.

Esser, P., Rombach, R., and Ommer, B. Taming transformers for high-resolution image synthesis. *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 12868–12878, 2020.

Federmann, C., Kocmi, T., and Xin, Y. NTREX-128 – news test references for MT evaluation of 128 languages. In *Proceedings of the First Workshop on Scaling Up Multilingual Evaluation*, pp. 21–24, Online, nov 2022. Association for Computational Linguistics.

Gandhi, S., von Platen, P., and Rush, A. M. Distil-whisper: Robust knowledge distillation via large-scale pseudo labelling. *CoRR*, abs/2311.00430, 2023. doi: 10.48550/ARXIV.2311.00430.

Gemma Team, Kamath, A., Ferret, J., Pathak, S., Vieillard, N., Merhej, R., Perrin, S., Matejovicova, T., Ramé, A., Rivière, M., Rouillard, L., Mesnard, T., Cideron, G., bastien Grill, J., Ramos, S., Yvinec, E., Casbon, M., Pot, E., Penchev, I., Liu, G., Visin, F., Kenealy, K., Beyer, L., Zhai, X., Tsitsulin, A., Busa-Fekete, R., Feng, A., Sachdeva, N., Coleman, B., Gao, Y., Mustafa, B., Barr, I., Parisotto, E., Tian, D., Eyal, M., Cherry, C., Peter, J.-T., Sinopalnikov, D., Bhupatiraju, S., Agarwal, R., Kazemi, M., Malkin, D., Kumar, R., Vilar, D., Brusilovsky, I., Luo, J., Steiner, A., Friesen, A., Sharma, A., Sharma, A., Gilady, A. M., Goedeckemeyer, A., Saade, A., Feng, A., Kolesnikov, A., Bendebury, A., Abdagic, A., Vadi, A., György, A., Pinto, A. S., Das, A., Bapna,

A., Miech, A., Yang, A., Paterson, A., Shenoy, A., Chakrabarti, A., Piot, B., Wu, B., Shahriari, B., Petrini, B., Chen, C., Lan, C. L., Choquette-Choo, C. A., Carey, C., Brick, C., Deutsch, D., Eisenbud, D., Cattle, D., Cheng, D., Paparas, D., Sreepathihalli, D. S., Reid, D., Tran, D., Zelle, D., Noland, E., Huizenga, E., Kharitonov, E., Liu, F., Amirkhanyan, G., Cameron, G., Hashemi, H., Klimczak-Plucińska, H., Singh, H., Mehta, H., Lehri, H. T., Hazimeh, H., Ballantyne, I., Szpektor, I., Nardini, I., Pouget-Abadie, J., Chan, J., Stanton, J., Wieting, J., Lai, J., Orbay, J., Fernandez, J., Newlan, J., yeong Ji, J., Singh, J., Black, K., Yu, K., Hui, K., Vodrahalli, K., Greff, K., Qiu, L., Valentine, M., Coelho, M., Ritter, M., Hoffman, M., Watson, M., Chaturvedi, M., Moynihan, M., Ma, M., Babar, N., Noy, N., Byrd, N., Roy, N., Momchev, N., Chauhan, N., Sachdeva, N., Bunyan, O., Botarda, P., Caron, P., Rubenstein, P. K., Culliton, P., Schmid, P., Sessa, P. G., Xu, P., Stanczyk, P., Tafti, P., Shivanna, R., Wu, R., Pan, R., Rokni, R., Willoughby, R., Vallu, R., Mullins, R., Jerome, S., Smoot, S., Girgin, S., Iqbal, S., Reddy, S., Sheth, S., Põder, S., Bhatnagar, S., Panyam, S. R., Eiger, S., Zhang, S., Liu, T., Yacovone, T., Liechty, T., Kalra, U., Evci, U., Misra, V., Roseberry, V., Feinberg, V., Kolesnikov, V., Han, W., Kwon, W., Chen, X., Chow, Y., Zhu, Y., Wei, Z., Egyed, Z., Cotruta, V., Giang, M., Kirk, P., Rao, A., Black, K., Babar, N., Lo, J., Moreira, E., Martins, L. G., Sanseviero, O., Gonzalez, L., Gleicher, Z., Warkentin, T., Mirrokni, V., Senter, E., Collins, E., Barral, J., Ghahramani, Z., Hadsell, R., Matias, Y., Sculley, D., Petrov, S., Fiedel, N., Shazeer, N., Vinyals, O., Dean, J., Hassabis, D., Kavukcuoglu, K., Farabet, C., Buchatskaya, E., Alayrac, J.-B., Anil, R., Dmitry, Lepikhin, Borgeaud, S., Bachem, O., Joulin, A., Andreev, A., Hardin, C., Dadashi, R., and Hussenot, L. Gemma 3 technical report, 2025.

Giorgino, T. Computing and visualizing dynamic time warping alignments in r: the dtw package. *Journal of statistical Software*, 31:1–24, 2009.

Graves, A. Sequence transduction with recurrent neural networks. *arXiv preprint arXiv:1211.3711*, 2012.

Graves, A., Mohamed, A., and Hinton, G. E. Speech recognition with deep recurrent neural networks. In *IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2013, Vancouver, BC, Canada, May 26-31, 2013*, pp. 6645–6649. IEEE, 2013. doi: 10.1109/ICASSP.2013.6638947.

Guo, S., Zhang, S., and Feng, Y. Decoder-only streaming transformer for simultaneous translation. *arXiv preprint arXiv:2406.03878*, 2024.

He, H., Shang, Z., Wang, C., Li, X., Gu, Y., Hua, H., Liu, L., Yang, C., Li, J., Shi, P., et al. Emilia: An extensive, multilingual, and diverse speech dataset for large-scale speech generation. In *2024 IEEE Spoken Language Technology Workshop (SLT)*, pp. 885–890. IEEE, 2024.

Hernandez, F., Nguyen, V., Ghannay, S., Tomashenko, N., and Esteve, Y. Ted-lium 3: Twice as much data and corpus repartition for experiments on speaker adaptation. In *Speech and Computer: 20th International Conference, SPECOM 2018, Leipzig, Germany, September 18–22, 2018, Proceedings 20*, pp. 198–208. Springer, 2018.

Ho, J. and Salimans, T. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022.

Hochreiter, S. and Schmidhuber, J. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.

Kang, W., Yang, X., Yao, Z., Kuang, F., Yang, Y., Guo, L., Lin, L., and Povey, D. Libriheavy: a 50,000 hours asr corpus with punctuation casing and context, 2023.

Kharitonov, E., Vincent, D., Borsos, Z., Marinier, R., Girgin, S., Pietquin, O., Sharifi, M., Tagliasacchi, M., and Zeghidour, N. Speak, read and prompt: High-fidelity text-to-speech with minimal supervision. *Transactions of the Association for Computational Linguistics*, 11:1703–1718, 2023.

Kreuk, F., Synnaeve, G., Polyak, A., Singer, U., Défossez, A., Copet, J., Parikh, D., Taigman, Y., and Adi, Y. Audiogen: Textually guided audio generation. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023.

Labiausse, T., Mazaré, L., Grave, E., Pérez, P., Défossez, A., and Zeghidour, N. High-fidelity simultaneous speech-to-speech translation, 2025.

Lee, D., Kim, C., Kim, S., Cho, M., and Han, W. Autoregressive image generation using residual quantization. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022*, pp. 11513–11522. IEEE, 2022. doi: 10.1109/CVPR52688.2022.01123.

Li, B., Gulati, A., Yu, J., Sainath, T. N., Chiu, C., Narayanan, A., Chang, S., Pang, R., He, Y., Qin, J., Han, W., Liang, Q., Zhang, Y., Strohman, T., and Wu, Y. A better and faster end-to-end model for streaming ASR. In *IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2021, Toronto, ON, Canada, June 6-11, 2021*, pp. 5634–5638. IEEE, 2021. doi: 10.1109/ICASSP39728.2021.9413899.

Liu, A. H., Ehrenberg, A., Lo, A., Denoix, C., Barreau, C., Lample, G., Delignon, J.-M., Chandu, K. R., von Platen, P., Muddireddy, P. R., Gandhi, S., Ghosh, S., Mishra, S., Foubert, T., Rastogi, A., Yang, A., Jiang, A. Q., Sablayrolles, A., Héliou, A., Martin, A., Agarwal, A., Roux, A., Darcet, A., Mensch, A., Bout, B., Rozière, B., Monicault, B. D., Bamford, C., Wallenwein, C., Renaudin, C., Lanfranchi, C., Dabert, D., Chaplot, D. S., Mizelle, D., de las Casas, D., Chane-Sane, E., Fugier, E., Hanna, E. B., Berrada, G., Delerce, G., Guinet, G., Novikov, G., Martin, G., Jaju, H., Ludziejewski, J., Rute, J., Chabran, J.-H., Chudnovsky, J., Studnia, J., Barmentlo, J., Amar, J., Roberts, J. S., Denize, J., Saxena, K., Yadav, K., Khandelwal, K., Jain, K., Lavaud, L. R., Blier, L., Zhao, L., Martin, L., Saulnier, L., Gao, L., Pellat, M., Guillaumin, M., Felardos, M., Dinot, M., Darrin, M., Augustin, M., Seznec, M., Gupta, N., Raghuraman, N., Duchenne, O., Wang, P., Saffer, P., Jacob, P., Wambergue, P., Kurylowicz, P., Chagniot, P., Stock, P., Agrawal, P., Delacourt, R., Sauvestre, R., Soletskyi, R., Vaze, S., Subramanian, S., Garg, S., Dalal, S., Gandhi, S., Aithal, S., Antoniak, S., Scao, T. L., Schueller, T., Lavril, T., Robert, T., Wang, T., Lacroix, T., Bewley, T., Nemychnikova, V., Paltz, V., Richard, V., Li, W.-D., Marshall, W., Zhang, X., Wan, Y., and Tang, Y. Voxtral, 2025. URL `https://arxiv.org/abs/2507.13264`.

Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*, 2019.

Louradour, J. whisper-timestamped. `https://github.com/linto-ai/whisper-timestamped`, 2023.

Lugosch, L., Ravanelli, M., Ignoto, P., Tomar, V. S., and Bengio, Y. Speech model pre-training for end-to-end spoken language understanding. *arXiv preprint arXiv:1904.03670*, 2019.

Ma, M., Huang, L., Xiong, H., Zheng, R., Liu, K., Zheng, B., Zhang, C., He, Z., Liu, H., Li, X., Wu, H., and Wang, H. STACL: Simultaneous translation with implicit anticipation and controllable latency using prefix-to-prefix framework. In Korhonen, A., Traum, D., and Màrquez, L. (eds.), *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 3025–3036, Florence, Italy, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1289. URL `https://aclanthology.org/P19-1289/`.

Ma, X., Wang, Y., Dousti, M. J., Koehn, P., and Pino, J. Streaming simultaneous speech translation with augmented memory transformer. In *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 7523–7527. IEEE, 2021.

Macháček, D., Dabre, R., and Bojar, O. Turning whisper into real-time transcription system. In Saha, S. and Sujaini, H. (eds.), *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics: System Demonstrations*, Bali, Indonesia, November 2023. Association for Computational Linguistics.

McAuliffe, M., Fatchurrahman, M. R., Feiteng, GalaxieT, NTT123, Gulati, A., Coles, A., Kong, C., Veaux, C., Eren, E., Gritskevich, E., Thor, G., Mishra, H., Fruehwald, J., Potrykus, P., Sereda, T., Mestrou, T., michaelasocolof, and vannawillerton. Montrealcorpustools/montreal-forced-aligner: Version 3.2.2, March 2025.

Oliveira, F. S., Casanova, E., Junior, A. C., Soares, A. S., and Galvão Filho, A. R. Cml-tts: A multilingual dataset for speech synthesis in low-resource languages. In Ekštein, K., Pártl, F., and Konopík, M. (eds.), *Text, Speech, and Dialogue*, pp. 188–199, Cham, 2023. Springer Nature Switzerland. ISBN 978-3-031-40498-6.

O'Neill, P. K., Lavrukhin, V., Majumdar, S., Noroozi, V., Zhang, Y., Kuchaiev, O., Balam, J., Dovzhenko, Y., Freyberg, K., Shulman, M. D., et al. Spgispeech: 5,000 hours of transcribed financial audio for fully formatted end-to-end speech recognition. *arXiv preprint arXiv:2104.02014*, 2021.

Panayotov, V., Chen, G., Povey, D., and Khudanpur, S. Librispeech: An ASR corpus based on public domain audio books. In *2015 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2015, South Brisbane, Queensland, Australia, April 19-24, 2015*, pp. 5206–5210. IEEE, 2015. doi: 10.1109/ICASSP.2015.7178964.

Radford, A., Narasimhan, K., Salimans, T., Sutskever, I., et al. Improving language understanding by generative pre-training. 2018.

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., and Sutskever, I. Robust speech recognition via large-scale weak supervision. In Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., and Scarlett, J. (eds.), *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, volume 202 of *Proceedings of Machine Learning Research*, pp. 28492–28518. PMLR, 2023.

Raffel, C., Luong, M., Liu, P. J., Weiss, R. J., and Eck, D. Online and linear-time attention by enforcing monotonic alignments. In Precup, D. and Teh, Y. W. (eds.), *Proceedings of the 34th International Conference on Machine Learning, ICML 2017, Sydney, NSW, Australia, 6-11 August 2017*, volume 70 of *Proceedings of Machine Learning Research*, pp. 2837–2846. PMLR, 2017.

Ramesh, A., Pavlov, M., Goh, G., Gray, S., Voss, C., Radford, A., Chen, M., and Sutskever, I. Zero-shot text-to-image generation. In Meila, M. and Zhang, T. (eds.), *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, volume 139 of *Proceedings of Machine Learning Research*, pp. 8821–8831. PMLR, 2021.

Razavi, A., van den Oord, A., and Vinyals, O. Generating diverse high-fidelity images with vq-vae-2. In Wallach, H., Larochelle, H., Beygelzimer, A., d'Alché-Buc, F., Fox, E., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.

Renals, S., Hain, T., and Bourlard, H. Recognition and understanding of meetings the ami and amida projects. In *2007 IEEE Workshop on Automatic Speech Recognition & Understanding (ASRU)*, pp. 238–247. IEEE, 2007.

Rio, M. D., Delworth, N., Westerman, R., Huang, M., Bhandari, N., Palakapilly, J., McNamara, Q., Dong, J., Zelasko, P., and Jette, M. Earnings-21: A practical benchmark for ASR in the wild. *CoRR*, abs/2104.11348, 2021.

Rubenstein, P. K., Asawaroengchai, C., Nguyen, D. D., Bapna, A., Borsos, Z., de Chaumont Quitry, F., Chen, P., Badawy, D. E., Han, W., Kharitonov, E., Muckenhirn, H., Padfield, D., Qin, J., Rozenberg, D., Sainath, T. N., Schalkwyk, J., Sharifi, M., Ramanovich, M. T., Tagliasacchi, M., Tudor, A., Velimirovic, M., Vincent, D., Yu, J., Wang, Y., Zayats, V., Zeghidour, N., Zhang, Y., Zhang, Z., Zilka, L., and Frank, C. H. Audiopalm: A large language model that can speak and listen. *CoRR*, abs/2306.12925, 2023. doi: 10.48550/ARXIV.2306.12925.

San Roman, R., Fernandez, P., Elsahar, H., D´efossez, A., Furon, T., and Tran, T. Proactive detection of voice cloning with localized watermarking. *ICML*, 2024.

Singh, M. K., Takahashi, N., Liao, W., and Mitsufuji, Y. SilentCipher: Deep Audio Watermarking. In *Proc. INTERSPEECH 2024*, 2024.

Srivastav, V., Majumdar, S., Koluguri, N., Moumen, A., Gandhi, S., et al. Open automatic speech recognition leaderboard. `https://huggingface.co/spaces/hf-audio/open_asr_leaderboard`, 2023.

Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

Sutskever, I., Vinyals, O., and Le, Q. V. Sequence to sequence learning with neural networks. *Advances in neural information processing systems*, 27, 2014.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., and and, L. K. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 5998–6008, 2017.

Wagner, L., Thallinger, B., and Zusag, M. Crisperwhisper: Accurate timestamps on verbatim speech transcriptions. *CoRR*, abs/2408.16589, 2024. doi: 10.48550/ARXIV.2408.16589.

Wang, C., Riviere, M., Lee, A., Wu, A., Talnikar, C., Haziza, D., Williamson, M., Pino, J., and Dupoux, E. Voxpopuli: A large-scale multilingual speech corpus for representation learning, semi-supervised learning and interpretation. *arXiv preprint arXiv:2101.00390*, 2021.

Wang, C., Chen, S., Wu, Y., Zhang, Z., Zhou, L., Liu, S., Chen, Z., Liu, Y., Wang, H., Li, J., He, L., Zhao, S., and Wei, F. Neural codec language models are zero-shot text to speech synthesizers. *CoRR*, abs/2301.02111, 2023. doi: 10.48550/ARXIV.2301.02111.

Wang, Y., Skerry-Ryan, R. J., Stanton, D., Wu, Y., Weiss, R. J., Jaitly, N., Yang, Z., Xiao, Y., Chen, Z., Bengio, S., Le, Q. V., Agiomyrgiannakis, Y., Clark, R., and Saurous, R. A. Tacotron: Towards end-to-end speech synthesis. In Lacerda, F. (ed.), *18th Annual Conference of the International Speech Communication Association, Interspeech 2017, Stockholm, Sweden, August 20-24, 2017*, pp. 4006–4010. ISCA, 2017. doi: 10.21437/INTERSPEECH.2017-1452.

Xu, H., Jia, F., Majumdar, S., Huang, H., Watanabe, S., and Ginsburg, B. Efficient sequence transduction by jointly predicting tokens and durations. In Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., and Scarlett, J. (eds.), *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, volume 202 of *Proceedings of Machine Learning Research*, pp. 38462–38484. PMLR, 2023.

Xue, J., Wang, P., Li, J., and Sun, E. A weakly-supervised streaming multilingual speech model with truly zero-shot capability. In *2023 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*, pp. 1–7. IEEE, 2023.

Yalniz, I. Z., Jégou, H., Chen, K., Paluri, M., and Mahajan, D. Billion-scale semi-supervised learning for image classification. *arXiv preprint arXiv:1905.00546*, 2019.

Yamagishi, J., Veaux, C., and MacDonald, K. Cstr vctk corpus: English multi-speaker corpus for cstr voice cloning toolkit (version 0.92), 2019.

Yu, L., Shi, B., Pasunuru, R., Muller, B., Golovneva, O. Y., Wang, T., Babu, A., Tang, B., Karrer, B., Sheynin, S., Ross, C., Polyak, A., Howes, R., Sharma, V., Xu, P., Tamoyan, H., Ashual, O., Singer, U., Li, S.-W., Zhang, S., James, R., Ghosh, G., Taigman, Y., Fazel-Zarandi, M., Celikyilmaz, A., Zettlemoyer, L., and Aghajanyan, A. Scaling autoregressive multi-modal models: Pretraining and instruction tuning. *ArXiv*, abs/2309.02591, 2023.

Zeghidour, N., Luebs, A., Omran, A., Skoglund, J., and Tagliasacchi, M. Soundstream: An end-to-end neural audio codec. *IEEE ACM Trans. Audio Speech Lang. Process.*, 30:495–507, 2022. doi: 10.1109/TASLP.2021.3129994.

Zelasko, P., Dhawan, K., Galvez, D., Puvvada, K. C., Pasad, A., Koluguri, N. R., Hu, K., Lavrukhin, V., Balam, J., and Ginsburg, B. Training and inference efficiency of encoder-decoder speech models. *CoRR*, abs/2503.05931, 2025. doi: 10.48550/ARXIV.2503.05931.

Zhang, L., Qian, Y., Zhou, L., Liu, S., Wang, D., Wang, X., Yousefi, M., Qian, Y., Li, J., He, L., et al. Covomix: Advancing zero-shot speech generation for human-like multi-talker conversations. *Proceedings of the 38th International Conference on Neural Information Processing Systems (NeurIPS 2024)*, 2024.

Zhang, Q., Lu, H., Sak, H., Tripathi, A., McDermott, E., Koo, S., and Kumar, S. Transformer transducer: A streamable speech recognition model with transformer encoders and rnn-t loss. In *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 7829–7833. IEEE, 2020.

A ASR Training Data

A.1 Short-form finetuning data

This collection includes: LibriSpeech (Panayotov et al., 2015), VoxPopuli (Wang et al., 2021), GigaSpeech (Chen et al., 2021a), AMI Carletta (2007); Renals et al. (2007), SPGISpeech (O'Neill et al., 2021), TED-LIUM (Hernandez et al., 2018), and Earnings22 (Del Rio et al., 2022). We filter out examples where Whisper had a high character error rate. With LibriSpeech (LS), we use timestamps provided by Montreal Forced Aligner (McAuliffe et al., 2025), as prepared by Lugosch et al. (2019). For the rest of the datasets, we use timestamps provided by Whisper (see Section 3.2). Importantly, at this stage, examples are short: only 0.5% of examples are longer than 30.5s. This data mixture totals 28k hours.

Table 9 summarizes licenses for the datasets used at this stage.

Dataset	License
AMI	CC-BY-4.0
EARNINGS22	CC-ShareAlike-4.0
GigaSpeech	Apache 2.0
LibriSpeech	CC-BY-4.0
SPGISpeech	Non-commercial use
TED-LIUM	BY-NC-ND 3.0
VoxPopuli	Creative Commons Zero

Table 9: Licenses for the public datasets we used for DSM-ASR finetuning.

A.2 Long-form finetuning data

First, we concatenate utterances in LibriSpeech (Panayotov et al., 2015) to form segments of up to 6 minutes long ($\approx$ 1k hours). Second, we use a collection of synthesized dialogs, each dialog lasting for 5 minutes ($\approx$ 22k hours). Those examples are produced by a preliminary version of DSM-TTS. We combine these datasets with the short-form finetuning dataset in a weighted mixture, with respective weights 8:1:1 (i.e., a sample from LibriSpeech is 8x more likely to be included in a batch than from the short-form data mixture).

B Size ablations for ASR

In the main text, we evaluated DSM-ASR model with 2.6B parameters. This model size might not be practical for all applications, so we additionally trained a smaller model with 300M parameters, using the same protocol, data, and vocabulary.

In Table 11, we compare its performance to several reference models. From this comparison, we see that DSM-ASR with 300M parameters (350M including Mimi encoder) achieves considerably better performance than streaming and non-streaming versions of *Whisper Medium*, which have 760M parameters. Unsurprisingly, the smaller version of DSM-ASR performs worse than a 2.6B version.

Finally, we note that these experiments suggest that the approach of DSM-ASR works well for models of smaller sizes, too.

C ASR: Efficiency as a function of batch size

In Table 10, we report how the efficiency metrics (RTF and throughput) change in function of the batch size used. In these measurements, we use the same DSM-ASR as in the main text.

Table 10: DSM-ASR: Real-time Factor and Throughput in function of batch size.

BATCH SIZE	RTF ↑	THROUGHPUT ↑
1	6.9	6.9
32	4.4	141.4
64	3.5	224.0
256	1.49	380.1

Table 11: **Short-form ASR performance across model sizes**. We report Word Error Rates (WER, %) for a DSM-ASR model with 300M parameters, along with reference models.

MODEL	AVG.	AMI	EARNINGS22	GIGASPEECH	LS CLEAN	LS OTHER	SPGISPEECH	TED-LIUM	VOXPOPULI
Non-streaming									
WHISPER MEDIUM.EN	8.1	16.7	12.6	11.0	3.0	5.9	3.3	4.1	9.6
Streaming									
WHISPER MEDIUM.EN	9.0	22.1	13.4	10.4	3.0	6.2	3.7	4.7	8.6
DSM-ASR 300M	8.2	16.4	13.9	11.1	2.2	6.8	2.7	4.2	8.4
DSM-ASR 2.6B	6.4	12.2	11.0	9.8	1.7	4.3	2.0	3.4	6.8

D Evaluation TTS Data

For evaluating our data, we use both monologue and dialog scripts. We do not have reference audio to compare to, although we can still compare models to one another. The datasets and evaluation code are available at github.com/kyutai-labs/tts_longeval.

Monologues. Those are taken from news articles of the NTREX-128 dataset (Federmann et al., 2022), giving us 123 articles per language. Note that those articles are already split in sentences, which we leverage when evaluating models that do not support long form generation.

Dialogs. Dialogs are generated using the `mistral-large-latest` model through their API[5]. The scripts fall in 3 categories: daily life, technical, and number-heavy. For daily life scripts, the LLM API is first asked to come up with a number of situations that could arise in daily interactions. For each one, it is then tasked with generating a script in a given language and with a target number of turns. We try to generate 50 daily dialogs per language (due to failures in the generation, only 97 in total are available). For technical topics, we follow a similar approach, except the LLM API is given a specific topic (technical, but also person, piece of art, movie, etc.) to discuss instead. We again have 50 technical dialogs per language. Finally, for number-heavy dialogs, we use 12 hand written prompts, such as "*PERSON_A and PERSON_B discuss the chronology of various events during the middle age*", which we use for generating scripts in both English and French. In total we thus have nearly 112 dialogs per language. Examples are provided in Figures 5 and 6.

Voices. For voice conditioning, we use samples from the test set of VCTK (Yamagishi et al., 2019) (ODC-BY license) for English, keeping only one utterance per speaker whose duration is longer than 7 seconds, and less than 10 seconds. We kept 50 voices. For French, we combine speakers from the test and valid sets of CML (Oliveira et al., 2023) (CC-BY 4.0), applying the same criterion. This gives us 35 voices. Pairs of voices are randomly assigned to each script, although the mapping is fixed across evaluation runs.

E Timestamp predictions

We adopt evaluation metrics from Wagner et al. (2024). Firstly, we compute an F1 score by defining true positives as words that match a reference word and overlap with it temporarily; false positive words are predicted but either do not match or do not overlap with a reference word; and false negatives are the reference words that do not have a match in content or have no temporal overlap with predicted words. The temporal overlap happens if the predicted timestamps for the word start and word end fall within a collar distance from the true events. Since this F1 metric considers overlap

[5]https://docs.mistral.ai/api/

Table 12: **Precision of predicted timestamps**. F1 and mIoU metrics, calculated on LS test-clean.

METRIC	CRISPERWHISPER	WHISPER LARGE-V3	DSM-ASR
F1	0.85	0.22	0.73
MIOU	0.65	0.30	0.54

in a binary way, Wagner et al. (2024) complement it with a more nuanced measure, mean Intersection over Union (mIoU), which additionally accounts for the relative duration of the temporal overlap. As the evaluation dataset, use the same time-aligned LibriSpeech test-clean as for latency measurement. Results are shown in Table 12.

F DSM-ASR: speech representation

One natural question is whether DSM-ASR is hindered by using discretized speech representation. In order to answer this question, we use the fact that the models are trained using quantizer dropout: at training time, after a randomly selected cut-off level, all quantization levels are zeroed out (Zeghidour et al., 2022; Défossez et al., 2024). We exploit this by running a series of evaluations of the same model on LibriSpeech test-clean (Panayotov et al., 2015), while systematically dropping quantizers at different cut-off levels. We observe that having fewer than 24 quantizers comes at a cost on WER; however, having more than that is not beneficial. Thus we conclude that having even more nuanced representation (e.g., continuous embeddings), might not lead to extra gains.

G TTS Baseline evaluations

We present how the baselines presented in Section 4.4 were evaluated. All baselines are given the same audio conditioning extracts of no more than 10 sec. for each speaker. The code for running the baselines is available at github.com/kyutai-labs/tts_longeval.

ElevenLabs. For monologues, we directly feed the entire script. For dialogs, we feed turns one by one, with no context (except for the speaker conditioning).

Dia. Dia was only trained to generate short audio extracts. For dialogs, we provide the turns used for speaker conditioning, and the last turn that was generated (e.g. from the other speaker than we are currently generating). We take care to provide them in such an order that speaker alternate. Even with limited context, generation sometimes failed, in which case we revert to generating the segment with no context (always keeping the speaker conditioning).

Orpheus. For monologues, we generate the sentences one by one with a context given by the speaker conditioning audio sample, along with one sentence of context. When that fails, we only give the audio conditioning sample. For dialogs, as Orpheus is single speaker, we generate the turns one by one, only with the speaker conditioning as context.

CSM. CSM normally only supports dialogs. For monologues, we have to repeat twice the same speaker in a row, which is out of distribution. We experimented with various parameters (deduplicating the speaker conditioning to pretend they are two speakers, different context size) and found the best results in terms of WER was obtained by keeping a single speaker conditioning, providing no extra past context, and still having two segments in a row with the same speaker id. For dialogs, we start with giving the turns used for speaker conditioning. Then we experimented with giving no context (short form), or giving up to 6 turns of context (long form). In case where the context would be too long and CSM raised an error, we would try again with a shorter context.

H Human evaluations

H.1 Models and dataset

The models are evaluated using the same parameters as described in Section G. For CSM, we only evaluate the version with no context which was giving the best WER. We use a subset of the data

provided in Section D, namely 15 monologues, and 9 dialogs (3 from each category) per language. For each script, we rate both the first 30 seconds, and the last 30 seconds separately.

H.2 Subjective evaluation tasks

Raters were payed on average 9.3£ per hour.

Quality. We also organize a MUSHRA style study for evaluating the audio quality, although with no explicit reference. For each script, the rater is presented with all the models' generations, and must note each one on a scale from 1 to 100. In particular, the instruction is as follows: "*How would you rate the overall quality of these audio clips? Consider aspects such as clarity, balance, richness, and naturalness*". We report the aggregated ratings, with the confidence intervals being the plus and minus the standard deviation of the mean estimator. For each language, 800 scripts were rated (each time covering all methods).

Speaker similarity. We present the rater with the audio used for speaker conditioning (either single speaker or the concatenation of both speakers for dialogs), along with two generations from two random models given the same script. The rater must choose which extract has speakers sound the most like the reference audio. The instruction is as follow: "*The reference contains the voices of one or two speakers. The audio files below it contain either a monologue or a dialog. Which voices sound more like the speakers in the reference?*" The win-rate is summarized as an *Elo score*, as described in the next paragraph. 1600 pairs were rated per language.

Bayesian Elo Score. The Elo score allows the ranking of models based on some pairwise comparisons of audio samples. Given two models A and B, the probability that A is preferred over B is:

$$P(A > B) = \frac{1}{1 + 10^{(E_B - E_A)/400}}, \tag{9}$$

where E_A and E_B are the Elo scores of each model. Unlike a traditional Elo score, the Bayesian Elo score uses a Gamma prior, so that one can derive confidence intervals over the posterior distribution.

We denote $S_A = 10^{(E_A - c)/400}$, with c freely chosen in $\mathbb{R}$. Then eq. (9) becomes

$$P(A > B) = \frac{S_A}{S_A + S_B}, \tag{10}$$

which is a Bradley-Terry (Bradley & Terry, 1952) model. We use the iterative by Caron & Doucet (2010) where S_A^0 follows a Gamma prior with parameters α^0, β^0. By denoting w_A the number of times where method A won against any other method and w_{AB} the number of times where A and B are compared, S_A^t is computed with the following update rule:

$$S_A^{t+1} = \frac{\alpha^0 + w_A}{\beta^0 + \sum_{B \neq A} \frac{w_{AB}}{S_A^t + S_B^t}}, \tag{11}$$

given as the mean of the Gamma distribution with updated parameters α_A^{t+1}, β_A^{t+1} given by

$$\alpha_A^{t+1} = \alpha^0 + w_A, \qquad \text{and} \qquad \beta_A^{t+1} = \beta^0 + \sum_{B \neq A} \frac{w_{A,B}}{S_A^t + S_B^t}. \tag{12}$$

Iterating over t allows reaching a fix point, we run 30 of them, once we have collected all the pairs. We use $\alpha = 0.1, \beta = 0.1, c = 2000$ so that, in absence of any data, $S_A = 1$ and $E_A = 2000$. Confidence intervals are 95% confidence interval according to the posterior.

I Latency, RTF, and throughput

We report in Table 6 latency, RTF, and throughput for various batch sizes for our model. Those numbers are obtained under real workload, with no synchronization between the batch elements, that is, requests for TTS arrive at the service at *random times*.

Batch size of 1. The latency represents the time to first audio, including all computations. Given that we use a delay $\tau = 2$ sec., or 25 steps, we need to account for the time that it takes to run those.

Table 13: **Resilience of existing watermarking techniques.** We compare the resilience of open-source watermarking techniques when applied to synthesized audio. After having applied the watermark, we encode and decode the audio using the Mimi codec (Défossez et al., 2024) with 32 RVQ levels. The Perth watermark was released at github.com/resemble-ai/perth.

Watermarking model	**Detection rate** (%)	**Detection rate after Mimi** (%)
AudioSeal (San Roman et al., 2024)	100.0%	45.2%
Resemble AI Perth	100.0%	0.0%
Silent Cipher (Singh et al., 2024)	82.3%	0.0%

We have to account for two times: d_b the duration it takes to run a single step of the backbone, and d_q the time it takes to run the small Transformer over the Q dimension. For the first 25 steps, we need only to run the backbone transformer, as no audio is produced. When the batch size is 1, the time to first token is given by $(\tau f_r - 1) \cdot d_b + (1 + 2) \cdot (d_b + d_q)$. The extra $2 \cdot (d_b + d_q)$ comes from the fact that, following Défossez et al. (2024), we use an *acoustic delay* of 2 steps between the first codebook and the remaining ones. With values of $d_b = 5$ ms, and $d_q = 25$ ms, we obtained the given results. The throughput and RTF are derived using only the time per step of $d_b + d_q$. Note that even though the small Transformer over Q is smaller in terms of dimensions and layers per codebook, it represents a significant amount of computation due to us using $Q = 32$ codebooks, leading to a large number of kernels being scheduled.

Interleaved steps for higher batch sizes. When needing to handle requests that are not in sync, with a larger batch size, we cannot just skip the small Transformer over Q, as not everyone will be at the same point in the generation. We use a 2-for-1 interleaving pattern, where if any of the batch items is in the initial phase, we interleave two steps of backbone only, followed by a full step. For items in the initial phase, this gives an effective time per step of $(d_b + 2 \cdot d_q)/3$.

J TTS results on LibriSpeech and Seed-en training on public datasets

We experiment with training DSM-TTS using only the publicly available datasets.

Architecture and training. This variant of the model uses a 300m backbone (24 layers with dimension 1024), $Q = 16$ codebooks. The small Transformer over Q is made more compact following (Labiausse et al., 2025), e.g. sharing weights for all codebooks from $[9 - 16]$, $[17, 24]$, along with using only 4 layers per step. We use a delay of $\tau = 1.28$ sec., or 16 steps. It receives no speaker conditioning, instead it has a probability of 20% of seeing the start of the audio with no text, and a probability of 10% of having this prefix completely empty, which will be used for applying classifier-free-guidance (Ho & Salimans, 2022; Kreuk et al., 2023). The model receives no text 20% of the time. The model is trained for 500k updates with a batch size of 64 on 16 H100 GPUs, using the same optimization parameters as described in Section 4.1.

Dataset We use a similar mixture as described in Section A.1, using the following datasets: AMI, EARNINGS22, GIGASpeech, SPGISpeech, TED-LIUM, VoxPopuli. Besides, we use Libri-Heavy (Kang et al., 2023) and Emilia (He et al., 2024). For LibriHeavy, we use the original formatted text. We filter out data with a high word error rate compared with Whisper transcripts. This totals to 88k hours of speech.

Evaluations We evaluate our approach on LibriSpeech test clean, with punctuation, following the same protocol as Chen et al. (2024b) (F5-TTS). When evaluating our model, we provide the speaker conditioning sample as a prefix. We use CFG factor $\alpha = 3$. We use the exact same evaluation set as F5-TTS, along with the same text normalization, ASR model (`whisper-large-v3`), and reuse their code for evaluating the speaker similarity to the original conditioning audio. We similarly evaluate on the Seed test en dataset (Anastassiou et al., 2024), following (Du et al., 2025). We provide support for those benchmarks in our evaluation codebase [6]. We release the $Q = 16$ version publicly[7]. This

[6] github.com/kyutai-labs/tts_longeval

[7] huggingface.co/kyutai/tts-0.75b-en-public

Table 14: Comparing DSM-TTS trained with Mimi and with EnCodec (Défossez et al., 2023). The number of RVQ levels in EnCodec was selected to roughly match the number of tokens per second with Mimi, keeping still a sufficient number to ensure decent quality. Training and evaluations are done as in Table 7.

Model	RVQ levels	Frame rate	Tokens per sec.	Bandwidth	WER	Speaker Sim.
F5-TTS	-	-	-	-	2.42 %	0.66
DSM-TTS with Mimi	32	12.5 Hz	400	4.4 kbps	1.68 %	0.71
DSM-TTS with Encodec	8	75 Hz	600	6 kbps	2.45 %	0.68

model allows for voice cloning through prefixing, although when generating turn-by-turn in the same setup as Table 4, with a score of 74.9%, against 80.9% for our main model. This level of speaker similarity is in line with some of the baselines like CSM and it thus seems safe to open source it.

Results Results are provided in Table 7. We achieve a large improvement of 0.3% of WER over the best F5-TTS model, however, with a small decrease in speaker similarity. While the diffusion based approach F5-TTS can generate faster a single script, it suffers from a higher latency, due to its non causal nature requiring the entire audio to be generated at once. Finally, we again show that the ease of batching of our methods allows for more than 100x throughput (duration of audio generated per second of computation) even for requests arriving unsynchronized, as explained in Section I.

Finally, we also provide results on the Seed test en dataset in Table 8. Our 16 codebooks model beats F5-TTS and is quite close to Cosyvoice 3-1.5B (RL) (Du et al., 2025), while being twice as small, and not requiring a reinforcement learning based fine tuning stage.

K On the efficacy of watermarking for TTS

We challenge the practice of relying on watermarking to limit potentially negative usage of TTS models. First, when open sourcing such a model, the watermarking stage can be disabled in the code. Besides, even if the watermark was built in the model, we noticed that most existing state of the art watermarking methods would almost completely disappear after a single round of encoding/decoding with Mimi with 32 codebooks, e.g. with minimal distortions. Results are provided in Table 13. Only AudioSeal (San Roman et al., 2024) is still detected half of the time. We believe this is because Mimi was not released at the time those watermark models were trained, and in particular, Mimi was trained with no spectrogram or waveform reconstruction loss, giving more freedom to how the input can be resynthesized, which the watermarking technique did not account for. This shows how fragile those approaches can be in the face of future innovation. The search for a reliable watermarking method remains an open and important question.

L Supplementary ablations on DSM-TTS

We now provide supplementary ablations.

First we compare training DSM-TTS with Mimi and with EnCodec (Défossez et al., 2023) in Table 14. Training and evaluation are performed as in Section J. We notice that while worse than with Mimi, our method generalizes well to other codecs.

Second, we study the impact of the lookahead and action stream presented in Section 3.3 in Table 15. We train and evaluate as in Section 4.4, either without a lookahead stream, or simply reusing the original DSM-TTS model, we ignore its action prediction and instead use a fixed padding strategy between words: we either force a fixed amount of padding after the start of the word, or a fixed amount of padding after its last text token. We notice that not using the lookahead has limited impact on the speaker similarity, but deteriorates the WER. On the other hand, using a fixed padding pattern has a clear impact on the speaker similarity, likely due to the fixed and unnatural prosody.

Table 15: We compare variants of DSM-TTS, in particular without a lookahead stream, and with a fixed padding strategy between words, e.g. without an action stream. For the fixed padding, we either force a fixed amount of padding after the start of the word, or a fixed amount of padding after its last text token. Training and evaluation is done as in Section 4.4.

Model variant	**WER English**	**WER French**	**Spk. Sim. English**	**Spk. Sim. French**
DSM-TTS baseline	1.60%	3.02%	0.743	0.745
No lookahead	3.51%	3.25%	0.743	0.746
Fixed padding, 4 from start	2.86%	3.60%	0.694	0.690
Fixed padding, 5 from start	2.69%	3.48%	0.691	0.700
Fixed padding, 2 from end	2.32%	3.13%	0.715	0.698

Dialog example, daily life

[LIU]: Hey Jean, how's it going with the studies?
[JEAN]: Not bad, Liu. Just trying to keep up with everything.
[LIU]: I hear you. Have you tried any study apps lately?
[JEAN]: Not really. I've been sticking to the old pen and paper method.
[LIU]: Oh, you should try this new app. It's really helpful.
[JEAN]: What's it called?
[LIU]: It's called "StudyBuddy." It helps you organize your notes and has flashcards too.
[JEAN]: Sounds interesting. How does it work?
[LIU]: You can upload your notes, create flashcards, and it even has a quiz feature.
[JEAN]: That sounds really useful. I'll give it a try.
[LIU]: Great! I think you'll find it really helpful.
[JEAN]: Alright, I'll download it tonight.
[LIU]: Awesome. Let me know how it goes.
[JEAN]: Will do. Thanks for the recommendation, Liu.
[LIU]: No problem, anytime.
[JEAN]: Hey, I just downloaded the app. It looks pretty good.
[LIU]: Glad to hear it. Have you tried any of the features yet?
[JEAN]: I uploaded some notes and made a few flashcards. It's really user-friendly.
[LIU]: That's great to hear. I find the quiz feature really helpful too.
[JEAN]: Yeah, I'm going to try that next. Thanks again, Liu.
[LIU]: You're welcome, Jean. Good luck with your studies!
[JEAN]: Thanks! Talk to you later.
[LIU]: Take care, Jean. See you in class!

Figure 5: Example of dialog generated with an LLM to serve as evaluation data. This dialog covers daily life topics.

Dialog example, daily life

[LUIS]: Mrs. Al-Falasi, have you heard of hydroxyapatite before?
[MRS AL-FALASI]: Yes, Luis. I know it's related to bones and teeth, but that's about it.
[LUIS]: Great start! Hydroxyapatite is a naturally occurring mineral form of calcium apatite. It has the formula Ca5(PO4)3(OH).
[MRS AL-FALASI]: That sounds quite complex. What makes it so important?
[LUIS]: It's the main mineral component of bones and teeth. It provides rigidity and strength.
[MRS AL-FALASI]: That makes sense. Is it used in any medical applications?
[LUIS]: Yes, synthetic hydroxyapatite is used in medical and dental applications for bone grafts and implants.
[MRS AL-FALASI]: Really? How does it work in those contexts?
[LUIS]: It has biocompatible properties, which means it's suitable for use in body tissue repair.
[MRS AL-FALASI]: That's fascinating. Are there any other uses for it?
[LUIS]: Absolutely. Hydroxyapatite can be used in coatings on orthopedic implants to promote bone growth.
[MRS AL-FALASI]: I didn't know that. It seems very versatile.
[LUIS]: It is. It's also used in toothpaste and mouthwash to help remineralize enamel.
[MRS AL-FALASI]: That explains why it's so beneficial for dental health. Anything else I should know?
[LUIS]: The material is effective in drug delivery systems, especially for bone diseases.
[MRS AL-FALASI]: That's impressive. Are there any non-medical uses?
[LUIS]: Yes, hydroxyapatite is utilized in chromatography for protein purification.
[MRS AL-FALASI]: Interesting. How about its sources? Where does it come from?
[LUIS]: It can be derived from natural sources such as animal bones or synthesized chemically.
[MRS AL-FALASI]: That's good to know. Any other applications I might not be aware of?
[LUIS]: Its porous structure makes it useful in filtering and ion-exchange applications.
[MRS AL-FALASI]: That's a lot of information. Thanks for explaining, Luis.
[LUIS]: You're welcome, Mrs. Al-Falasi. Hydroxyapatite is truly a remarkable material.

Figure 6: Example of dialog generated with an LLM to serve as evaluation data. This dialog covers technical topics.